

Next.js Interview Questions & Answers

Technical Questions

1. Explain the differences between SSR, SSG, ISR, and CSR in Next.js.

Server-Side Rendering (SSR):

- Generates HTML on the server for each request
- Implemented via `getServerSideProps()`
- Best for pages that need fresh data on every request
- Provides better SEO than CSR
- Slower TTFB (Time To First Byte) compared to SSG
- Example use case: Dashboard showing real-time data

Static Site Generation (SSG):

- Generates HTML at build time
- Implemented via `getStaticProps()`
- Best for pages where content rarely changes
- Fastest page loads as content is pre-rendered
- Can be served from CDN for optimal performance
- Example use case: Blog posts, documentation

Incremental Static Regeneration (ISR):

- Combines benefits of SSG and SSR
- Implemented via `getStaticProps()` with `revalidate` property
- Pages are generated statically but revalidated after specified time
- Updates stale pages in the background
- Example use case: E-commerce product pages with occasional price/inventory updates

Client-Side Rendering (CSR):

- Initial HTML is minimal, content rendered by JavaScript in browser
- Implemented with `useEffect()` and `fetch` or `SWR/React Query`
- Best for highly interactive, private, or frequently changing content
- Worse for SEO (though Next.js provides workarounds)
- Example use case: User-specific content, interactive applications

The key is choosing the right rendering strategy based on your specific page requirements. Next.js allows you to mix and match these approaches on a per-page basis.

2. How do you optimize image loading and performance in Next.js?

Next.js provides the optimized `Image` component (`next/image`) which offers several performance benefits:

1. Automatic image optimization:

- Converts images to modern formats like WebP and AVIF
- Resizes images to prevent shipping large images to small screens
- Lazy loads images by default (loads only when they enter viewport)

2. Implementation best practices:

jsx

 Copy

```
import Image from 'next/image';

function MyComponent() {
  return (
    <Image
      src="/profile.jpg"
      width={500}
      height={300}
      placeholder="blur"
      blurDataURL="data:image/jpeg;base64,..."
      priority={true} // for above-the-fold images
      alt="Profile picture"
    />
  );
}
```

3. Advanced optimizations:

- Use `priority` attribute for above-the-fold images
- Implement blur placeholders for better perceived performance
- Configure domains in `next.config.js` for external images
- Use responsive sizes with the `sizes` attribute
- Implement proper CDN configuration

4. Configuration in `next.config.js`:

```
module.exports = {
  images: {
    domains: ['example.com'], // For external images
    formats: ['image/avif', 'image/webp'],
    deviceSizes: [640, 750, 828, 1080, 1200, 1920, 2048],
    imageSizes: [16, 32, 48, 64, 96, 128, 256, 384],
  },
}
```

5. When to use static imports:

```
import profilePic from '../public/profile.jpg';

function MyComponent() {
  return (
    <Image
      src={profilePic}
      alt="Profile picture"
      placeholder="blur" // Automatic blurDataURL with static imports
    />
  );
}
```

By leveraging these features, you can significantly improve Core Web Vitals metrics like LCP (Largest Contentful Paint) and CLS (Cumulative Layout Shift).

3. Explain Next.js API routes and when you would use them.

Next.js API Routes allow you to create serverless API endpoints within your Next.js application:

1. Basic implementation:

- Create files in the `/pages/api` directory
- Each file becomes an API endpoint

```
// pages/api/hello.js
export default function handler(req, res) {
  res.status(200).json({ message: 'Hello World!' });
}
```

2. When to use API routes:

- When you need to keep sensitive operations server-side (API keys, authentication)

- For server-side form submission handling
- Creating BFF (Backend for Frontend) patterns
- Proxying external API requests to avoid CORS issues
- Implementing webhooks
- Building prototype backends quickly
- Serverless functions without managing separate infrastructure

3. **Advanced patterns:**

- Dynamic API routes with `[param].js` or `[...slug].js`
- API middleware for authentication or logging
- Error handling and proper status codes
- Rate limiting and security best practices

4. **Limitations:**

- Not suitable for long-running processes
- Subject to serverless cold starts and execution time limits
- Cannot use server-only libraries that rely on Node.js globals

5. **When NOT to use API routes:**

- For very high traffic APIs (consider dedicated API service)
- For complex microservices architecture
- When you need websocket support (though workarounds exist)

With Next.js 13's App Router, Server Actions provide an alternative to some API route use cases, allowing you to collocate server-side logic directly with your components.

4. **How do you handle authentication in Next.js?**

Authentication in Next.js can be implemented in several ways:

1. **NextAuth.js (recommended approach):**

- Most popular authentication library for Next.js
- Supports OAuth providers, credentials, email/password
- Handles sessions, JWT, CSRF protection
- Works well with both Pages Router and App Router

```
// pages/api/auth/[...nextauth].js
import NextAuth from "next-auth";
import GithubProvider from "next-auth/providers/github";

export default NextAuth({
  providers: [
    GithubProvider({
      clientId: process.env.GITHUB_ID,
      clientSecret: process.env.GITHUB_SECRET,
    }),
  ],
  callbacks: {
    async session({ session, token }) {
      // Add custom session logic
      return session;
    },
  },
});
```

2. Middleware-based authentication:

- Protect routes using Next.js middleware

```
// middleware.js
export function middleware(req) {
  const { pathname } = req.nextUrl;
  const isAuthenticated = checkAuthStatus(); // Your auth check

  if (!isAuthenticated && pathname !== '/login') {
    return NextResponse.redirect(new URL('/login', req.url));
  }
}

export const config = {
  matcher: ['/dashboard/:path*', '/profile/:path*'],
}
```

3. Server component/Server action authentication (App Router):

- Authentication state available directly in server components
- Use server actions for login/logout operations
- No client-side hydration issues

4. SSR authentication with `getServerSideProps` (Pages Router):

```
export async function getServerSideProps(context) {
  const session = await getSession(context);

  if (!session) {
    return {
      redirect: {
        destination: '/login',
        permanent: false,
      },
    };
  }

  return {
    props: { session },
  };
}
```

5. Client-side authentication:

- Using a library like Auth0, Firebase Auth, etc.
- Checking auth state in useEffect
- Protected routes with HOC pattern or React Context

6. Common authentication patterns:

- JWT stored in HTTP-only cookies
- Role-based access control
- Session refresh tokens
- Remember-me functionality

The trend is moving toward server-centric authentication flows that handle tokens securely server-side, especially with the App Router's server components architecture.

5. Describe your experience with Next.js middleware.

Next.js middleware runs before a request is completed, allowing you to modify the response based on the incoming request:

1. Core capabilities:

- Route matching and filtering
- Rewriting, redirecting, or modifying requests
- Setting headers and cookies
- Authentication and authorization checks

- A/B testing and feature flags
- Logging and analytics

2. Implementation:

javascript

 Copy

```
// middleware.js (root of project)
import { NextResponse } from 'next/server';

export function middleware(request) {
  // Get the pathname
  const { pathname } = request.nextUrl;

  // Check if user has a specific cookie
  const hasCookie = request.cookies.has('my-cookie');

  // Redirect to login if accessing protected route without cookie
  if (pathname.startsWith('/dashboard') && !hasCookie) {
    return NextResponse.redirect(new URL('/login', request.url));
  }

  // Add custom headers
  const response = NextResponse.next();
  response.headers.set('x-custom-header', 'my-value');

  return response;
}

// Optional: configure paths middleware runs on
export const config = {
  matcher: ['/dashboard/:path*', '/api/:path*'],
}
```

3. Advanced use cases:

- Internationalization (i18n) routing
- Paywall or subscription access control
- Bot detection and prevention
- Edge environment geolocation-based content
- Request throttling/rate limiting
- Response compression

4. Performance considerations:

- Middleware runs on the Edge network for better performance
- Should be lightweight and fast

- Avoid heavy processing or database calls

5. Limitations:

- Limited Node.js API access (runs in Edge runtime)
- No access to the full request body
- Cannot directly modify the HTML response

Middleware is powerful for cross-cutting concerns that need to be applied consistently across routes without duplicating logic in individual page components.

6. How do you implement internationalization in Next.js?

Internationalization (i18n) in Next.js can be implemented in several ways:

1. Built-in i18n support (Pages Router):

javascript

 Copy

```
// next.config.js
module.exports = {
  i18n: {
    locales: ['en', 'fr', 'de'],
    defaultLocale: 'en',
    localeDetection: true, // automatic locale detection
  },
}
```

Then access locale info in pages:

javascript

 Copy

```
export async function getStaticProps({ locale }) {
  return {
    props: {
      messages: await import(`../locales/${locale}.json`),
    },
  };
}
```

2. App Router internationalization:

- Use route groups with folders like `[lang]`
- Create middleware for locale detection and routing


```
// middleware.js
export function middleware(request) {
  const { pathname } = request.nextUrl;
  const pathnameHasLocale = locales.some(
    locale => pathname.startsWith(`/${locale}/`) || pathname === `/${locale}`
  );

  if (!pathnameHasLocale) {
    // Redirect to defaultLocale
    const locale = getBestLocaleFromHeader(request.headers.get('accept-language'));
    request.nextUrl.pathname = `/${locale}${pathname}`;
    return NextResponse.redirect(request.nextUrl);
  }
}
```

3. Translation libraries:

- **next-intl**: Full-featured i18n library optimized for Next.js
- **next-i18next**: Popular solution for Pages Router
- **react-intl**: More general React i18n solution

Example with next-intl:

```
import { useTranslations } from 'next-intl';

function MyComponent() {
  const t = useTranslations('Common');
  return <h1>{t('welcome')}</h1>;
}
```

4. Advanced i18n patterns:

- **Message formatting** for plurals, dates, numbers
- **Right-to-left (RTL) support** with CSS direction
- **Date/time/number formatting** based on locale
- **Currency display** based on user region
- **Content translation workflows** with CMS integration

5. Performance considerations:

- Use dynamic imports for translation files
- Consider static generation with locale paths
- Implement proper caching strategies

6. SEO for internationalized sites:

- Set proper `<html lang>` attribute
- Implement `<link rel="alternate" hreflang="x">` tags
- Add locale-specific meta tags
- Use localized sitemaps

The key is balancing developer experience, performance, and comprehensive i18n features based on your project's specific needs.

7. Explain the Next.js App Router vs. Pages Router.

The App Router and Pages Router represent two architectural approaches in Next.js:

Pages Router (Traditional):

- File-based routing in `/pages` directory
- Each `.js`, `.jsx`, `.ts`, or `.tsx` file becomes a route
- Uses special functions like `getStaticProps`, `getServerSideProps`
- Client-side rendering by default with optional server rendering
- Data fetching happens at build/request time
- Uses `_app.js` and `_document.js` for global layouts

App Router (Modern):

- Introduced in Next.js 13
- File-based routing in `/app` directory
- Uses a more nested folder structure with special files (`page.js`, `layout.js`, etc.)
- Server Components by default for improved performance
- Client Components opt-in with "use client" directive
- Data fetching happens in the component itself
- Parallel and streaming rendering capabilities
- Built-in support for nested layouts

Key differences:

1. Component model:

- Pages Router: Client Components by default
- App Router: Server Components by default, more efficient rendering

2. Data fetching:

- Pages Router: Uses `getServerSideProps`, `getStaticProps`

- App Router: Data fetching directly in components, supports fetch with built-in caching

3. Routing:

- Pages Router: Simpler file structure, special filename patterns
- App Router: More conventions, folder-based grouping, parallel routes

4. Layouts:

- Pages Router: Uses HOCs or custom layout components
- App Router: Built-in nested layouts with `layout.js`

5. Special files:

- Pages Router: `_app.js`, `_document.js`, etc.
- App Router: `layout.js`, `loading.js`, `error.js`, etc.

6. API Routes:

- Pages Router: `/pages/api` directory
- App Router: `/app/api` directory, plus Server Actions for form submissions

7. Performance:

- App Router generally has better performance potential due to:
 - Automatic code splitting
 - Partial rendering
 - Streaming SSR
 - More efficient client/server boundary

Both routers can coexist in the same project for gradual migration. The App Router represents Next.js's future direction, focusing on React Server Components and improved performance.

8. How do you deploy Next.js applications?

Next.js applications can be deployed in several ways:

1. Vercel (recommended):

- Created by the same team behind Next.js
- Zero-configuration deployment
- Automatic preview deployments for PRs
- Edge functions for middleware
- Integrated analytics and performance monitoring
- Global CDN with edge caching

bash

 Copy

```
# Install Vercel CLI
npm i -g vercel
# Deploy
vercel
```

2. Netlify:

- Similar to Vercel with preview deployments
- Integrated forms and functions
- Simple setup with Netlify adapter

javascript

 Copy

```
// next.config.js
const { withNitric } = require('@netlify/next');
module.exports = withNitric({
  // your Next.js config
});
```

3. Self-hosted (Node.js server):

- Build the application: `next build`
- Start the production server: `next start`
- Usually deployed behind Nginx or Apache
- Consider PM2 for process management

4. Docker containerization:

```
FROM node:18-alpine AS base

# Install dependencies
FROM base AS deps
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci

# Build the app
FROM base AS builder
WORKDIR /app
COPY --from=deps /app/node_modules ./node_modules
COPY . .
RUN npm run build

# Production image
FROM base AS runner
WORKDIR /app
COPY --from=builder /app/public ./public
COPY --from=builder /app/.next/standalone ./
COPY --from=builder /app/.next/static ./next/static

EXPOSE 3000
ENV PORT 3000

CMD ["node", "server.js"]
```

5. Static export:

- For sites without server-side features
- Use `next.config.js`:

javascript

 Copy

```
module.exports = {
  output: 'export',
}
```

- Run `next build` to generate `/out` directory
- Deploy to any static hosting (AWS S3, GitHub Pages, etc.)

6. Cloud providers:

- **AWS:** Elastic Beanstalk, ECS, or Lambda + API Gateway
- **Google Cloud:** Cloud Run or App Engine

- **Azure:** App Service or Container Instances

7. Deployment considerations:

- Environment variables management
- CI/CD pipeline setup
- Health checks and monitoring
- CDN configuration for static assets
- Database and backend service connectivity
- Containerization for consistent environments

8. Optimizing for production:

- Enable compression
- Configure proper caching headers
- Use a CDN for static assets
- Implement proper error logging
- Set up performance monitoring

The optimal deployment strategy depends on your application's requirements, expected traffic, and DevOps capabilities.

9. What strategies do you use for loading states and error handling?

Loading States:

1. App Router approach:

- Use `loading.js` files for automatic suspense boundaries

jsx

 Copy

```
// app/dashboard/loading.js
export default function DashboardLoading() {
  return <div className="loading-spinner">Loading dashboard...</div>;
}
```

- Implement skeleton screens for better UX

jsx

 Copy

```
export default function ProductSkeleton() {
  return (
    <div className="skeleton-card">
      <div className="skeleton-image pulse"></div>
      <div className="skeleton-title pulse"></div>
      <div className="skeleton-price pulse"></div>
    </div>
  );
}
```

2. Pages Router approach:

- Use React Suspense with fallback

jsx

 Copy

```
<Suspense fallback=<LoadingSkeleton />>
  <ProductDetails id={productId} />
</Suspense>
```

- Implement custom loading states with React Query or SWR

jsx

 Copy

```
const { data, error, isLoading } = useSWR('/api/user', fetcher);

if (isLoading) return <LoadingSpinner />;
if (error) return <ErrorMessage error={error} />;
return <UserProfile data={data} />;
```

3. Progressive loading patterns:

- Stream content as it becomes available
- Load critical content first
- Defer non-critical content loading
- Use priority hints for important resources

Error Handling:

1. App Router approach:

- Use `error.js` files for automatic error boundaries

```
// app/dashboard/error.js
'use client';

export default function DashboardError({ error, reset }) {
  return (
    <div className="error-container">
      <h2>Something went wrong loading the dashboard</h2>
      <p>{error.message}</p>
      <button onClick={() => reset()}>Try again</button>
    </div>
  );
}
```

- Use `not-found.js` for 404 errors

```
// app/products/[id]/not-found.js
export default function ProductNotFound() {
  return (
    <div>
      <h2>Product not found</h2>
      <p>The product you're looking for doesn't exist.</p>
    </div>
  );
}
```

2. Pages Router approach:

- Custom error pages: `pages/404.js`, `pages/500.js`
- React Error Boundaries


```
class ErrorBoundary extends React.Component {
  constructor(props) {
    super(props);
    this.state = { hasError: false, error: null };
  }

  static getDerivedStateFromError(error) {
    return { hasError: true, error };
  }

  render() {
    if (this.state.hasError) {
      return <ErrorComponent error={this.state.error} />;
    }
    return this.props.children;
  }
}
```

3. API error handling:

- Consistent error response format

```
// pages/api/products.js
export default function handler(req, res) {
  try {
    // Process request
    res.status(200).json({ data: products });
  } catch (error) {
    console.error(error);
    res.status(500).json({
      error: {
        message: 'Internal server error',
        details: process.env.NODE_ENV === 'development' ? error.message : undefined
      }
    });
  }
}
```

- Error logging and monitoring integration

4. Global error handling:

- Error monitoring tools (Sentry, LogRocket)
- Custom hooks for error processing

```
function useErrorHandler() {
  const [error, setError] = useState(null);

  const handleError = useCallback((error) => {
    // Log to monitoring service
    logErrorToService(error);
    // Set local error state
    setError(error);
  }, []);

  return { error, handleError, clearError: () => setError(null) };
}
```

The best approach combines proper error boundaries with graceful fallbacks and helpful error messages to maintain a good user experience even when things go wrong.

10. How do you optimize SEO in Next.js applications?

Next.js provides several built-in features and best practices for SEO optimization:

1. Metadata API (App Router):

```
// app/products/[id]/page.js
export const generateMetadata = async ({ params }) => {
  const product = await getProduct(params.id);

  return {
    title: product.name,
    description: product.description,
    openGraph: {
      images: [{ url: product.image }],
    },
  };
};
```

2. Head Component (Pages Router):

```
import Head from 'next/head';

function ProductPage({ product }) {
  return (
    <>
      <Head>
        <title>{product.name}</title>
        <meta name="description" content={product.description} />
        <meta property="og:image" content={product.image} />
      </Head>
      { /* Page content */ }
    </>
  );
}
```

3. Server-side rendering for search engines:

- Use SSR or SSG for important landing pages
- Ensures content is available when search engines crawl
- Better Core Web Vitals scores

4. Structured data (JSON-LD):

```
<script
  type="application/ld+json"
  dangerouslySetInnerHTML={{
    __html: JSON.stringify({
      "@context": "https://schema.org",
      "@type": "Product",
      name: product.name,
      image: product.image,
      description: product.description,
      offers: {
        "@type": "Offer",
        price: product.price,
        priceCurrency: "USD"
      }
    })
  }}
/>
```

5. Optimized images:

- Use Next.js Image component with proper alt text

- Prevents layout shifts that hurt SEO
- Improves page load time

6. Canonical URLs:

jsx

 Copy

```
<link
  rel="canonical"
  href={`https://example.com/products/${product.id}`}
/>
```

7. XML Sitemaps and robots.txt:

- Dynamic sitemap generation in `/pages/sitemap.xml.js`
- Custom robots.txt in `/pages/robots.txt.js`
- Alternatively, place static files in `/public`

8. Internationalization:

- Implement proper hreflang tags
- Use localized URLs and content

9. Performance optimization:

- Optimize Core Web Vitals (LCP, FID, CLS)
- Enable caching where appropriate
- Implement proper font loading strategy

10. Additional best practices:

- Create SEO-friendly URLs
- Implement breadcrumbs navigation
- Add semantic HTML elements
- Ensure mobile responsiveness
- Use descriptive anchor text for links
- Implement proper header hierarchy

11. SEO monitoring and testing:

- Implement Google Search Console integration
- Test with Lighthouse and PageSpeed Insights
- Monitor SEO performance with analytics

By combining these techniques, Next.js applications can achieve excellent SEO performance while maintaining good user experience.

11. Explain how you'd implement a custom server in Next.js.

While Next.js recommends using its built-in server capabilities, there are scenarios where a custom server might be necessary:

1. Basic custom server setup:

javascript

 Copy

```
// server.js
const { createServer } = require('http');
const { parse } = require('url');
const next = require('next');

const dev = process.env.NODE_ENV !== 'production';
const app = next({ dev });
const handle = app.getRequestHandler();

app.prepare().then(() => {
  createServer((req, res) => {
    const parsedUrl = parse(req.url, true);
    const { pathname, query } = parsedUrl;

    // Custom routing logic
    if (pathname === '/a') {
      app.render(req, res, '/b', query);
    } else {
      handle(req, res, parsedUrl);
    }
  }).listen(3000, (err) => {
    if (err) throw err;
    console.log('> Ready on http://localhost:3000');
  });
});
```

2. When to use a custom server:

- Implementing server-side redirects
- Handling custom routing not covered by Next.js
- Using middleware not compatible with Edge Runtime
- Running additional services alongside Next.js
- Implementing WebSockets
- Integrating with existing Express/Koa/Hapi applications

3. Express.js integration:

```
// server.js
const express = require('express');
const next = require('next');

const dev = process.env.NODE_ENV !== 'production';
const app = next({ dev });
const handle = app.getRequestHandler();

app.prepare().then(() => {
  const server = express();

  // Custom middleware
  server.use(express.json());
  server.use(cookieParser());

  // Custom routes
  server.get('/products/:id', (req, res) => {
    return app.render(req, res, '/product', { id: req.params.id });
  });

  // Default handler
  server.all('*', (req, res) => {
    return handle(req, res);
  });

  server.listen(3000, (err) => {
    if (err) throw err;
    console.log('> Ready on http://localhost:3000');
  });
});
```

4. Deployment considerations with custom server:

- Need to run Node.js directly, not compatible with static exports
- Potentially higher hosting costs vs. serverless deployment
- Manual scaling and management required
- Package the server code with the Next.js application

5. Performance optimizations:

- Implement appropriate caching
- Use clustering for multi-core utilization

```
// server.js with clustering
const cluster = require('cluster');
const numCPUs = require('os').cpus().length;

if (cluster.isMaster) {
  console.log(`Master ${process.pid} is running`);

  // Fork workers
  for (let i = 0; i < numCPUs; i++) {
    cluster.fork();
  }

  cluster.on('exit', (worker) => {
    console.log(`Worker ${worker.process.pid} died`);
    cluster.fork(); // Replace the dead worker
  });
} else {
  // Workers can share any TCP connection
  // In this case it is an HTTP server
  const app = next({ dev: false });
  const handle = app.getRequestHandler();

  app.prepare().then(() => {
    createServer((req, res) => {
      // Your custom server logic
      handle(req, res);
    }).listen(3000);

    console.log(`Worker ${process.pid} started`);
  });
}
```

6. Alternatives to custom server:

- Use Next.js middleware (preferred for most use cases)
- Use redirects in `next.config.js`
- Use rewrites in `next.config.js`
- Use API routes for backend functionality
- Use edge functions for global distribution

While custom servers still have use cases, Next.js has evolved to provide built-in solutions for many scenarios that previously required custom servers. Always evaluate if you can use Next.js's native features before implementing a custom server.

12. Describe your experience with Server Components and Server Actions.

React Server Components (RSC):

1. Core benefits:

- Render complex components on the server with zero client-side JavaScript
- Reduced bundle size and improved performance
- Direct database/filesystem access without API layer
- Automatic code splitting
- Streaming rendering capabilities

2. Implementation in Next.js:

- Default in App Router (all components are server components unless marked with "use client")
- Components can access server-only resources directly

jsx

 Copy

```
// app/users/page.js (Server Component)
import { sql } from '@vercel/postgres';

export default async function UsersPage() {
  // Direct database access on the server
  const { rows } = await sql`SELECT * FROM users`;

  return (
    <div>
      <h1>Users</h1>
      <ul>
        {rows.map(user => (
          <li key={user.id}>{user.name}</li>
        ))}
      </ul>
    </div>
  );
}
```

3. Data fetching patterns:

- Fetch directly in server components
- Parallel data fetching for performance


```
async function Dashboard() {  
  // These requests run in parallel  
  const userData = getUser();  
  const productsData = getProducts();  
  const revenueData = getRevenue();  
  
  // Wait for all promises to resolve  
  const [user, products, revenue] = await Promise.all([  
    userData, productsData, revenueData  
  ]);  
  
  return (  
    // Render with all data available  
  );  
}
```

4. Client/Server composition:

- Clear boundary management with "use client" directive
- Passing server data to client components as props
- Interleaving server and client components for optimal performance

```
// ClientComponent.js
'use client';

import { useState } from 'react';

export default function ClientComponent({ serverData }) {
  const [clientState, setClientState] = useState(false);

  return (
    <div>
      <h2>Server provided data: {serverData.title}</h2>
      <button onClick={() => setClientState(!clientState)}>
        Toggle client state: {clientState ? 'ON' : 'OFF'}
      </button>
    </div>
  );
}

// ServerComponent.js
import ClientComponent from './ClientComponent';
import { getServerData } from '../lib/data';

export default async function ServerComponent() {
  const data = await getServerData();

  return (
    <div>
      <h1>Server Component</h1>
      <ClientComponent serverData={data} />
    </div>
  );
}
```

5. Performance optimization techniques:

- Component-level caching with React cache

```
import { cache } from 'react';

export const getUser = cache(async (userId) => {
  const user = await db.user.findUnique({ id: userId });
  return user;
});
```

- Strategic client/server component boundaries
- Streaming for faster Time To First Byte (TTFB)

Server Actions:

1. Core benefits:

- Server-side form handling and data mutations
- Progressive enhancement (works without JavaScript)
- Built-in CSRF protection
- Optimistic updates with error handling
- Revalidation of data after mutations

2. Implementation:

jsx

 Copy

```
// app/actions.js
'use server';

import { revalidatePath } from 'next/cache';

export async function createTodo(formData) {
  const title = formData.get('title');

  // Input validation
  if (!title || title.length < 3) {
    return { error: 'Title must be at least 3 characters' };
  }

  try {
    // Save to database
    await db.todo.create({ data: { title } });

    // Revalidate the todos page
    revalidatePath('/todos');
    return { success: true };
  } catch (error) {
    return { error: 'Failed to create todo' };
  }
}
```

3. Usage in forms:

```
import { createTodo } from './actions';

export default function TodoForm() {
  return (
    <form action={createTodo}>
      <input name="title" placeholder="New todo" />
      <button type="submit">Add Todo</button>
    </form>
  );
}
```

4. Advanced patterns:

- Optimistic updates with useOptimistic

```
'use client';

import { useOptimistic } from 'react';
import { createTodo } from './actions';

export default function TodoList({ todos }) {
  const [optimisticTodos, addOptimisticTodo] = useOptimistic(
    todos,
    (state, newTodo) => [...state, newTodo]
  );

  async function handleSubmit(formData) {
    const title = formData.get('title');

    // Add optimistic todo
    addOptimisticTodo({ id: 'temp-id', title, status: 'pending' });

    // Actual server action
    await createTodo(formData);
  }

  return (
    <div>
      <form action={handleSubmit}>
        <input name="title" />
        <button type="submit">Add Todo</button>
      </form>
      <ul>
        {optimisticTodos.map(todo => (
          <li key={todo.id}>{todo.title}</li>
        ))}
      </ul>
    </div>
  );
}
```

- Server action composition
- Form validation with error handling
- Progressive enhancement support

5. Security considerations:

- CSRF protection built-in
- Input validation on the server

- Rate limiting for mutation actions
- Proper error handling and logging

Experience-Based Questions

1. Describe a complex Next.js application you've built.

When describing a complex Next.js application I've built, I would structure my answer to demonstrate both technical depth and business impact:

"I led the development of an enterprise-level e-commerce platform serving over 500,000 monthly users. The application had several technical challenges that made Next.js an ideal choice:

Technical architecture highlights:

1. Hybrid rendering strategy:

- Product listing pages used ISR with a 1-hour revalidation period, balancing freshness with performance
- Product detail pages used SSG for top 1,000 products and on-demand ISR for the long tail
- Shopping cart and checkout used client-side rendering with aggressive caching strategies
- This approach reduced server costs by 40% while maintaining sub-second Time to First Byte

2. International multi-tenant solution:

- Implemented a dynamic multi-region, multi-language architecture
- Used Next.js middleware to handle tenant detection, language routing, and authentication
- Custom locale resolution based on user preferences, URL structure, and geolocation
- Shared component library that adapted to tenant-specific styling and functionality

3. Performance optimization:

- Implemented module federation for shared components across multiple Next.js applications
- Reduced JavaScript bundle size by 60% through aggressive code splitting and tree shaking
- Implemented a service worker strategy for offline capabilities and resource caching
- Achieved 98+ Lighthouse performance scores across critical user journeys

4. Integration architecture:

- Built a BFF (Backend for Frontend) layer using Next.js API routes
- Implemented data aggregation across multiple microservices
- Created a real-time inventory and pricing system using WebSockets
- Developed a robust error handling and retry mechanism for unreliable third-party APIs

Outcome and impact:

- 40% improvement in conversion rate due to performance optimizations
- 25% reduction in bounce rate on product pages
- Successful scaling during Black Friday with 300% normal traffic without degradation
- Platform now powers 12 regional storefronts across 8 languages

The project showcased Next.js's flexibility in handling diverse rendering requirements within a single application while maintaining excellent performance and developer experience."

2. How have you leveraged Next.js features to improve user experience?

"I've consistently leveraged Next.js features to create exceptional user experiences across various projects. Here's how I've applied specific Next.js capabilities to solve real UX challenges:

1. **Image optimization for visual stability:** In a media-heavy portfolio site for a photography client, I implemented the Next.js Image component with proper sizing, format optimization, and placeholder blurring. This eliminated layout shifts (improving CLS by 87%) and reduced LCP by 2.3 seconds on 3G connections. The visual stability significantly improved user engagement metrics, with average session duration increasing by 45%.

jsx

 Copy

```
<Image
  src={photo.url}
  width={photo.width}
  height={photo.height}
  placeholder="blur"
  blurDataURL={photo.blurDataUrl}
  priority={index < 3}
  alt={photo.description}
/>
```

2. **Streaming and suspense for perceived performance:** For a data-heavy dashboard application, I implemented streaming SSR with suspense boundaries to dramatically improve perceived performance. Users would see the dashboard shell and navigation within 300ms while data-intensive widgets would stream in progressively. This reduced perceived load time by 70% according to user testing, despite the complete page taking the same total time to load.

```
// app/dashboard/page.js
export default function Dashboard() {
  return (
    <DashboardShell>
      <Suspense fallback={<UserProfileSkeleton />}>
        <UserProfile />
      </Suspense>
      <Suspense fallback={<RevenueChartSkeleton />}>
        <RevenueChart />
      </Suspense>
      {/* Additional components */}
    </DashboardShell>
  );
}
```

- 3. App Router and partial rendering:** When migrating a complex application from Pages Router to App Router, I focused on leveraging partial rendering to enhance user experience during navigation. By properly structuring layouts and implementing nested routing, we achieved a 60% reduction in unnecessary re-renders during navigation. This made transitions between pages feel instant, even on lower-end devices.
- 4. Server actions for form handling:** In a membership application system, I replaced client-side form submissions with server actions. This simplified the code and improved reliability by eliminating client-side validation discrepancies. Most importantly, it improved accessibility by ensuring forms worked without JavaScript and reduced form abandonment rates by 15% on slower connections.
- 5. Route handlers for dynamic pricing:** For an event ticketing platform, I implemented route handlers to provide real-time dynamic pricing directly from the server. This eliminated the "price changed" errors that previously frustrated users by ensuring prices displayed were always current. The solution improved booking completion rates by 23%.

The key to my approach has been focusing on how Next.js features can solve specific user experience problems rather than implementing them for technical reasons alone. Each solution was measured against real user metrics to validate the improvements."

3. What challenges have you faced when migrating from React to Next.js?

"Migrating from React to Next.js presents several challenges that I've successfully navigated across multiple projects. Here's an overview of the key challenges I've encountered and how I addressed them:

- 1. Mental model shift from CSR to SSR/SSG** The most fundamental challenge was helping the team shift their mental model from client-side rendering to server-first thinking. Developers

accustomed to React's client-side paradigm often struggled with concepts like hydration, server components, and the implications of the build step. **Solution:** I created an incremental migration strategy that began with a simple Next.js wrapper around the existing React application. We then gradually migrated routes, starting with static content pages where SSG benefits were most obvious. I also ran workshops focusing on server/client boundaries and data flow patterns, using visual diagrams to clarify the new mental models.

2. **Global state management restructuring** The existing React application relied heavily on global state management (Redux) for everything, including data that would be better served through server rendering in Next.js. **Solution:** We audited all Redux usage and categorized state into:

- Server-renderable data (moved to `getServerSideProps` or `getStaticProps`)
- Transient UI state (moved to local `useState` or `useReducer`)
- True global client state (kept in Redux but minimized)

This reduced our client bundle size by 35% and simplified the application architecture.

3. **Handling browser-specific APIs** The React application had many components that directly accessed browser APIs (`window`, `document`, `localStorage`) without checks, causing SSR errors.

Solution: I implemented a pattern of conditional imports and runtime checks:

jsx

 Copy

```
// utils/storage.js
export const getItem = (key) => {
  if (typeof window !== 'undefined') {
    return localStorage.getItem(key);
  }
  return null;
};
```

For more complex cases, we created a `useClientOnly` hook to safely handle browser-only code.

4. **CSS-in-JS compatibility** The application used a CSS-in-JS library (Emotion) that wasn't configured for SSR, causing styling mismatches between server and client renders. **Solution:** We added proper Next.js configuration for Emotion and implemented a custom `Document` component to ensure styles were properly extracted during SSR. For new components, we gradually migrated to CSS Modules for better performance and simplicity.

5. **Routing and navigation patterns** Transitioning from React Router to Next.js's file-based routing required rethinking navigation patterns and URL structures. **Solution:** We mapped out all existing routes and created a migration plan that preserved URLs where possible. We implemented a compatibility layer with redirects for changed routes and added route guards to replicate protected route patterns from the React application.

6. **Build-time vs. runtime environment variables** In the React app, all environment variables were available at runtime, but Next.js distinguishes between build-time and runtime variables. **Solution:**

We reorganized environment variables following Next.js conventions, using NEXT_PUBLIC_ prefixes appropriately and implementing a runtime configuration API for truly dynamic values.

The migration yielded significant benefits: 62% improvement in First Contentful Paint, 45% reduction in Total Blocking Time, and vastly improved SEO performance. The key lesson was that incremental migration with clear patterns and extensive testing at each stage was far more successful than attempting a complete rewrite."